

# Resumo FTC - Prova I

Sophia  
Carvazza



## Matéria que vai cair:

- 2 - Introdução: preliminares matemáticas
- 3 - Exercícios
- 4 - Linguagens: conjuntos e expressões regulares
- 5 - Exercícios
- 6 - Autômatos
- 7 - Exercícios
- 8 - Autômatos e Linguagens: pumping lemma
- 9 - Exercícios
- 10 - Gramáticas

# Preliminares Matemáticas

## > Relação Binária

↳ uma forma matemática de expressar qualquer tipo de correspondência entre conjuntos:

$$R \subseteq A \times B \quad \text{↗ (subconjunto do produto cartesiano } A \times B)$$

- > Reflexiva:  $\forall x \in A [x R x] \quad (\forall a \in A: (a, a) \in R)$
- > Simétrica:  $\forall x, y \in A [x R y \rightarrow y R x]$ 
  - ↳  $(\forall a, b \in A: (a, b) \in R \Rightarrow (b, a) \in R)$
- > Transitiva:  $\forall x, y, z \in A [(x R y \cap y R z) \rightarrow x R z]$ 
  - ↳  $(\forall a, b, c \in A: (a, b) \in R \cap (b, c) \in R \Rightarrow (a, c) \in R)$

## > Funções

\* Funções:

parcial ↗ (f de A para B)  
- parcial:  $f: A \rightarrow B$  é uma relação binária  $f \subseteq A \times B$  tal que se  $(x, y) \in f$  e  $(x, z) \in f$ , então  $y = z$ .

↳ uma função parcial não está definida p/ alguns valores do seu conjunto inicial (ex:  $\frac{1}{x}$ , não existe  $\frac{1}{0}$ , então a função não está definida em zero).

↳ nem todo valor de entrada tem saída garantida.

- total:  $f: A \rightarrow B$  é uma função parcial definida p/ todo elemento  $x \in A$ .

↳ um função total está definida p/ todos os valores do conjunto inicial (ex: a soma sobre os reais,  $+R \rightarrow R$ , daí a divisão não é total, por causa de 0).

↳ qualquer valor de entrada do domínio terá uma saída.

- injetora: se para  $x, y \in A$  quaisquer,  $x \neq y \rightarrow f(x) \neq f(y)$   
↳ a função recebe uma única saída diferente para o mesmo resultado (ex:  $f(x) = x + 1$ ).

↳ entradas diferentes dão saídas diferentes.

- Sobrejetora: se  $B$  é a imagem de  $f$ .

↳ se todo elemento do conjunto de chegada é atingido por alguma saída (ex:  $f(x) = x^3$ ).

↳ "correspondência um para um"  
- Bijetora: se é injetora e sobrejetora. Cada elemento do domínio corresponde a um único elemento da Imagem (ex:  $f(x) = x + 2$ ).

# Sintaxe e Semântica

↳ disposição dos palavras na frase e a relação lógica dos frases entre si.

- reconhecido antes da semântica
- tratamento + simples

→ estuda o significado dos palavras na frase e das frases no discurso

- analisado após a sintaxe
- tratamento + elaborado e subjetivo

## > linguagem formal

- alfabeto
- cadeia de caracteres / palavras

## > alfabeto

- conjunto finito de símbolos ou caracteres
- notação:  $\Sigma$

→ entidade abstrata básica

→ são alfabetos =  $\{a, b, c\}$ ,  $\emptyset$  (conjunto vazio)

→ não são alfabetos =  $\mathbb{N}$  (conjunto dos naturais),  $\{a, bb, cc, \dots\}$

↳ é infinito

indica que não é finito

## > palavra

- sequência finita de símbolos justapostos ou caracteres
- cadeia de caracteres; sentença
- notação:  $w$

↳ ex em  $\{a, b\} \Rightarrow \sqrt{\Rightarrow ab, bb, aa, \varepsilon \text{ (cadeia vazia)}}$   
 $\Leftrightarrow x \Rightarrow ab\dots, aaa\dots, abc$

\* concatenação  $\Rightarrow$  operação binária sobre um conjunto de palavras

- associa duas palavras

- é a justaposição da 1ª com a 2ª.

→ é associativa, então podemos sempre trocar os parenteses da posição.

$$v(wt) = (vw)t \quad \leftarrow$$



- Elemento neutro das palavras:  $\varepsilon w = w = w \varepsilon$

↳ concatenação sucessiva

-  $b^4 = b b b b$     -  $a^0 = \varepsilon$     -  $a^3 b^5 = a a a b b b b b$

↳ se  $\Sigma$  é um alfabeto:

- $\Sigma^*$  é o conjunto de todas as palavras possíveis sobre  $\Sigma$
- $\Sigma^+ = \Sigma^* - \varepsilon$

> linguagem formal

- é um subconjunto de  $\Sigma^*$
- ou seja, é um subconjunto de todas "palavras" possíveis dentro de um alfabeto.
- notação:  $L$

↳ ex:  $\emptyset \in \{\varepsilon\}$ ,  $\Sigma^*$  e  $\Sigma^+$  são linguagens sobre qualquer alfabeto

• comum

> gramática

↳ é uma quádrupla  $G = (V, \Sigma, P, S)$

$V$  = conjunto de símbolos não-terminais ou variáveis (ex:  $S, A, B$ )

$\Sigma$  = alfabeto  $\rightarrow$  conjunto de símbolos terminais (ex:  $a, b, c$ )

$P$  = conjunto de produções ou regras  $P \subseteq (V \cup \Sigma)^+ \times (V \cup \Sigma)^+$

$S$  = símbolo inicial ou variável de partida ( $S \in V$ )

> expressão regular

↳ é uma notação que representa um conjunto regular (linguagem regular)

↳ sua definição recursiva é:

- BASE

- P.R (passo recursivo)

- FECHAMENTO

ex:  $(a \cup b)^* = \{\varepsilon, a, b, aa, ab, aac, \dots\}$

## > linguagem regular

- ↳ é uma linguagem  $L$  regular se e somente se:
  - existe um autômato finito que a reconhece (AFD, AFN ou AFN- $\lambda$ )
  - deve possuir uma expressão regular que a descreva.
  - ex regular:  $\{w \mid w \text{ termina com } 01\}$
  - não regular:  $\{0^n 1^n \mid n \geq 0\}$  (nem n. de Q.e ts)
    - ↳ provamos com bombeamento

## DEFINIÇÕES RECURSIVAS

- ↳ Todos os números naturais são construídos a partir de 0 e da função de sucessor  $S$ .

### - definição do conjunto dos naturais $\mathbb{N}$

$$\mathbb{N}: \begin{cases} \text{base: } 0 \in \mathbb{N} \\ \text{P.R.: se } n \in \mathbb{N} : S(n) \in \mathbb{N} \end{cases} \quad \text{sucessor}$$

### - definição de sucessor

$$S(n): \begin{cases} \text{base: } S: \mathbb{N} \rightarrow \mathbb{N}, \\ \text{P.R.: } S(n) = n + 1 \end{cases}$$

Sem usar o operador  $+$ :

$$\begin{cases} \text{soma}(a, 0) = a \\ \text{soma}(a, S(b)) = S(\text{soma}(a, b)) \end{cases}$$

### - definição de soma

$$+: \begin{cases} \text{base: } a + 0 = a \\ \text{P.R.: } a + c = a + S(b) = S(a + b) \end{cases}$$

### - definição de multiplicação

$$\cdot: \begin{cases} \text{base: } a \cdot 0 = 0 \\ \text{P.R.: } a \cdot c = a \cdot S(b) = a + a \cdot b \end{cases}$$

$$\underbrace{3 \cdot 4}_{12} = 3 \cdot \underbrace{S(3)}_{(4)} = 3 + \underbrace{3 \cdot 3}_{9} = 12$$

- definição de exponenciação (potência)

base:  $a^0 = S(0) \rightarrow$  (ou  $\epsilon$ ,  $\perp$ )

P.R.:  $a^{S(b)} = (a^b) \cdot a$

$$2^3 = 2^{S(2)} = 2^2 \cdot 2$$

1. Dê a definição recursiva do conjunto de strings sobre o alfabeto  $\{a, b\}$  que contenha um número par de b's.

$\{$  base:  $\epsilon \in L, a \in L$   
P.R.: - se  $w \in L$ :  $\begin{cases} wa \in L \\ aw \in L \end{cases}$

- se  $w \in L$ :  $\begin{cases} wb \in L \\ bw \in L \end{cases}$

- se  $w_1, w_2 \in L$ :  $w_1 b b w_2 \in L$

+ dois b's  
mantém a  
paridade par.

se  $w \in L$ , já tem  
qntd par de b's, então  
odd. a's não altera

se  $w \notin L$ , a qntd  
de b's é ímpar, então  
adicionando + uma  
fica par.

# > Autômatos:

## > sistemas de estados finitos

- modelo matemático de sistema
- comporta por entradas e estados
- não tem memória
- tem um n° finito e predeterminado de estados
- a máquina só pode estar em um estado por vez

## > tipos:

### determinístico

- a partir de um determinado estado e do símbolo lido, pode assumir um único estado.

### não-determinístico

- a partir de um determinado estado e do símbolo lido, pode assumir um conjunto de estados.

### com mov. vazias

- a partir de um determinado estado e sem ler um símbolo, pode assumir um conjunto de estados.

## ★ Autômato Finito Determinístico (AFD)

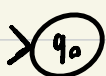
- ↳ possui um único estado de transição por cada par de estado e símbolo de entrada, resultando em um caminho único e fixo.

↳  $M = (\Sigma, Q, \delta, q_0, F)$

$\Sigma$ : alfabeto  
 $Q$ : conjunto de estados possíveis do autômato  
 $\delta$ : função de transição  
 $q_0$ : estado inicial  
 $F$ : subconjunto de  $Q$  (conjunto dos estados finais)

$$\delta: Q \times \Sigma \rightarrow Q$$

estado inicial



estado final



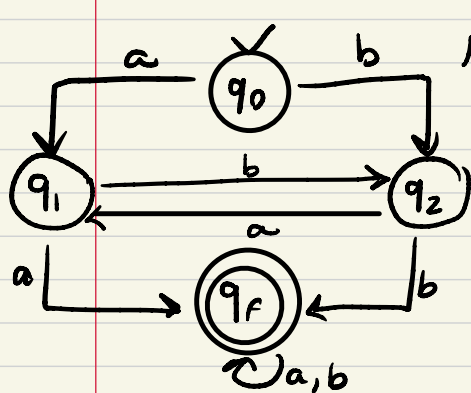
função de transição por tabela  $\rightarrow$  ex:  $\delta(p, a) = q$

| $\delta$ | a   | ... |
|----------|-----|-----|
| p        | q   | ... |
| q        | ... | ... |

exemplo:

$\Sigma = \{a, b\}$  que possui como subpalavra aa ou bb

$L = \{w \mid w \text{ possui } \underline{aa} \text{ ou } \underline{bb} \text{ como subpalavra}\}$



$M_1 = (\{a, b\}, \{q_0, q_1, q_2, q_f\}, \delta, q_0, \{q_f\})$

| $\delta$ | a     | b     |
|----------|-------|-------|
| $q_0$    | $q_1$ | $q_2$ |
| $q_1$    | $q_f$ | $q_0$ |
| $q_2$    | $q_1$ | $q_f$ |
| $q_f$    | $q_f$ | $q_f$ |

**Determinismo Incompleto** = quando nem todas as transições são especificadas (a função de  $\delta$  é parcial)

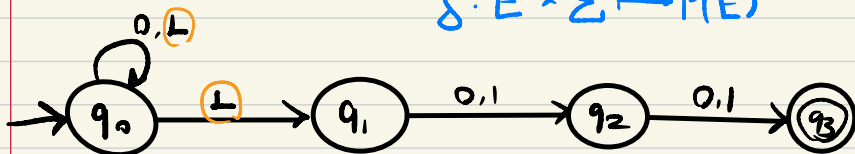
★ Para toda linguagem finita existe um AFD!!!  
 $\rightarrow$  ele pode ser não mínimo!

## \* Autômato Finito Não Determinístico (AFN)

↳ permitem + de uma transição partindo de um estado com o mesmo símbolo do alfabeto.

↳  $M = (E, \Sigma, \delta, i, F)$  ↗ conjunto de estados finais

conjunto de estados  $E$  alfabeto de entrada  $\Sigma$  estado inicial  $i$  função de transição  $\delta: E \times \Sigma \rightarrow P(E)$



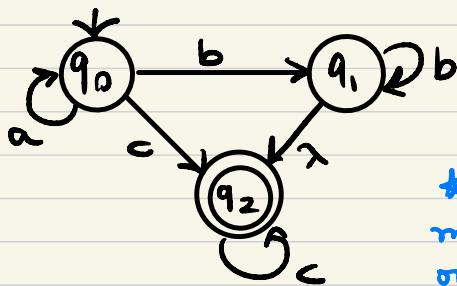
↳ uma palavra pode gerar diferentes computações!

AFNE  
ou

## \* Autômato Finito com Movimentos Vazios (AFN- $\lambda$ )

↳ é não determinístico, e possui transição lambda ( $\lambda$ ) ou epsilon ( $\epsilon$ ) vazia, que permitem que o autômato mude de estado sem consumir nenhum símbolo.

- generaliza os movimentos não-determinísticos
- transição sem leitura, vazia (encapsulada).



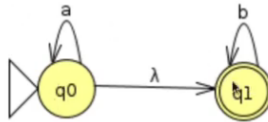
$M = (\Sigma, Q, \delta, q_0, F)$

\* em uma transição vazia, assumimos simultaneamente o estado origem e destino!

- $L = \{ w \mid w \text{ possui } a\text{'s que antecedem } b\text{'s} \}$
- $AFN_{\epsilon}$ 
  - Matriz de transição

| $\delta$ | a    | b    | $\epsilon$ |
|----------|------|------|------------|
| q0       | {q0} | -    | {q1}       |
| q1       | -    | {q1} | -          |

Exemplo de  $AFN-\lambda$



## TRANSFORMAÇÕES

### ↳ $AFN-\lambda \rightarrow AFN$

(remoção da transição vazia)

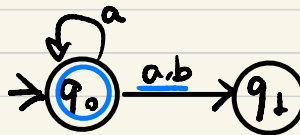
p/ todo  $AFN-\lambda$ , existe um AFN.

★ passo a passo:

- analisamos tanto o nó origem quanto o destino, p/ cada símbolo do alfabeto.
- substituímos o vazio por todos os símbolos do alfabeto (pq pode ser qualquer um deles)

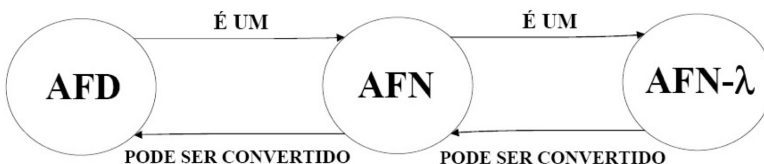
nesso mesmo exemplo:

|    | a    | b    | $\lambda$ |
|----|------|------|-----------|
| q0 | {q0} | { }  | {q1}      |
| q1 | { }  | {q1} | { }       |



se não substituímos o vazio, a palavra nula não é aceita.

↳ devemos analisar cada caso específico



## ↳ AFN $\rightarrow$ AFD

★ passo a passo:

1º: tabela de transição AFN

2º: tabela de transição AFD

↳ estados = todas as possíveis combinações do AFN

↳ total de estados =  $2^n$

↳ quando um estado for um conjunto, o novo será a união dos transições

3º: marcar os novos estados finais

↳ todos os conjuntos que contêm algum estado final do AFN

4º: eliminar os estados:

↳ inalcançáveis

↳ que não possuem saídas e não são finais

5º: verificar no grafo se há algum ciclo inalcançável.

## AFN to AFD

### Exemplo

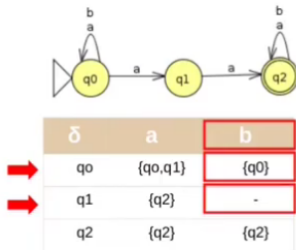


Tabela de transição do AFN

| $\varnothing$ | a            | b    |
|---------------|--------------|------|
| {q0}          | {q0, q1}     | {q0} |
| {q1}          | {q2}         | -    |
| {q2}          | {q2}         | {q2} |
| {q0, q1}      | {q0, q1, q2} | {q0} |
| {q0, q2}      |              |      |
| {q1, q2}      |              |      |
| {q0, q1, q2}  |              |      |
| $\emptyset$   |              |      |

Tabela de transição do AFD

### Estado {q0, q1}

- São dois estados, logo será a união deles
- AFN, q0 ao processar "b" vai para {q0}
- AFN, q1 ao processar "b" vai para -  
{q0}  $\cup$  - = {q0}



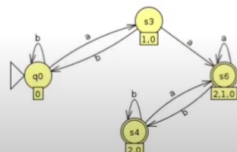
## Exemplo



| $\delta$ | a       | b    |
|----------|---------|------|
| q0       | {q0,q1} | {q0} |
| q1       | {q2}    | -    |
| q2       | {q2}    | {q2} |

Tabela de transição do AFN

|     | $\varphi$  | a  | b  |
|-----|------------|----|----|
| S0  | {q0}       | S3 | S0 |
| S3  | {q0,q1}    | S6 | S0 |
| *S4 | {q0,q2}    | S6 | S3 |
| *S6 | {q0,q1,q2} | S6 | S4 |



|                | $\varphi$          | a             | b             |
|----------------|--------------------|---------------|---------------|
| S0             | {q0}               | S3            | S0            |
| <del>S1</del>  | <del>{q1}</del>    | <del>S2</del> | <del>-</del>  |
| <del>*S2</del> | <del>{q2}</del>    | <del>S2</del> | <del>S2</del> |
| S3             | {q0,q1}            | S6            | S0            |
| *S4            | {q0,q2}            | S6            | S4            |
| <del>*S5</del> | <del>{q1,q2}</del> | <del>S2</del> | <del>S2</del> |
| *S6            | {q0,q1,q2}         | S6            | S4            |
| <del>S7</del>  | <del>-</del>       | <del>-</del>  | <del>-</del>  |

- Qual(is) era(m) final(is) no AFN?
- R: q2
- Agora basta marcar no AFD todos que tem esses estados, eles serão os novos estados finais

- Eliminar estados inalcançáveis

- Na prática é eliminar os estados que não aparecem nas colunas "a" e "b" de outro estado
- Ou seja, ninguém chega nele

- eliminar os estados que não possuem saída e não são finais (loops)

Expressão Regular a Partir de um Autômato:

I -  $\circ \xrightarrow{\alpha} \circ \xrightarrow{\beta} \circ \Rightarrow \circ \xrightarrow{\alpha\beta} \circ$

II -  $\circ \xrightarrow{\alpha} \circ \xrightarrow{\beta} \circ \xrightarrow{\delta} \circ \Rightarrow \circ \xrightarrow{\alpha\beta^*\delta} \circ$

III -  $\circ \xrightarrow{\alpha} \circ \xrightarrow{\beta} \circ \Rightarrow \circ \xrightarrow{\alpha+\beta} \circ$

# Lema do Bombeamento

Em um autômato finito, um trecho de cadeia pode ser repetido em um loop várias vezes ("bombeado") sem nunca levar o autômato a um estado não aceitante - sem tirar a palavra da linguagem.

Analogamente, quando suspeitamos que uma linguagem  $L$  não pertence às regulares (para loops infinitos), utilizamos esse lema como prova por contradição.

→ (ou absurdo)?

## ★ U L E M A :

Seja  $L$  uma linguagem regular. Então existe um inteiro  $K \geq 1$  (comprimento de bombeamento) tal que, p/ toda palavra  $w \in L$  com  $|w| \geq K$ , podemos escrever:

$$w = pqv$$

ex:  $w = 0011$

$$w = \lambda \mid 00 \mid 11$$

$p \mid p \mid v$

Satisfazendo:

i)  $w = pqv$

ii)  $|pq| \leq K$

iii)  $|q| \neq \lambda$  ( $|q| \neq 0$ )

iv)  $w_i = pq^i v \in L \quad \forall i \geq 0$  → (tem que continuar fazendo parte da linguagem)

★  $|w| \geq K$  pq a palavra deve passar por um loop para fazermos o lema!